

Automation Tool 사용 가이드

HECTO PASS QA Automation Tool 사용 설명서

문서 정보

- 담당자 및 문의: QA 3팀 김태림 (회선 정보 추가/변경 및 오류 문의)
- 도구 버전: ver 0.85
- 배포 경로: 사내 Google Drive 배포 폴더 (기능 추가 및 개선 시 지속 업데이트)
- 지원 환경: Windows 10 / 11, Google Chrome 브라우저 설치 필수 PC

1. 개요 및 도입 배경

본 도구는 PC Web을 통해 접근하는 Test Page의 접근을 최소화하여 테스트 효율을 극대화하기 위해 제작되었습니다.

반복적인 파라미터 수기 입력과 상용/QA 도메인 전환의 번거로움을 해결하고, 사용자의 편의성을 강화하기 위해 **단일 실행 파일(.exe)** 하나로 모든 검증을 수행할 수 있도록 설계되었습니다.

구분	기존 Web 수기 테스트 방식	QA Automation Tool 적용 방식
작업 공수	메뉴 이동 시마다 반복적인 폼 데이터 수기 입력	사용자 선택 시 회선 및 파라미터 원클릭 자동 주입
환경 파편화	QA 도메인, 상용 IP 접속 경로를 수기로 전환	버튼 하나로 QA/상용 환경 및 전용 도메인 자동 라우팅
브라우저 제어	웹 브라우저 조작에 따른 창 간섭 및 렌더링 부하	백그라운드(Headless) 실행 지원으로 화면 간섭 제로화
복합 검증	2단계 점프, 가변 QR 렌더링, 공개키 검증 수기 진행	단일 톨 내 비동기 처리 및 실시간 결과 JSON 파싱 제공

2. 사전 준비 및 배포 안내

1. 사전 필수 조건:

- PC 내 **Google Chrome** 브라우저가 설치되어 있어야 합니다 (Selenium 웹 제어 엔진 구동용).

2. 배포 및 업데이트:

- 본 도구는 단일 실행 파일(**HECTO_QA_Tool.exe**)로 제공됩니다.

- 신규 기능 추가 및 개선 버전은 사내 **Google Drive**를 통해 지속적으로 업로드 및 배포됩니다.
- 상단 타이틀 영역의 **Version** 라벨을 클릭하면 업데이트 노트를 확인할 수 있습니다.

3. 실행 시 주의사항:

- 백그라운드에서 브라우저 프로세스가 기동되므로 최초 실행 시 2~3초간 준비 시간이 필요합니다.
- 정상 구동을 위해 **.exe** 파일은 반드시 **1회만 실행**해 주세요 (중복 실행 금지).

3. 기본 워크플로우 (Quick Start)

Step 0: 테스트 환경 및 모드 선택

- 테스트 환경 선택:
 - **QA 환경:** QA 스테이지 도메인 자동 연결
 - **상용 환경(IP):** 상용 도메인 정책/방화벽 이슈를 대응하기 위해 지정된 상용 IP로 자동 라우팅
- 백그라운드 실행 (Web 미노출):
 - **ON (기본 권장):** 웹 창을 띄우지 않고 백그라운드에서 빠르고 조용하게 동작합니다.
 - **OFF (디버그 모드):** 실제 Chrome 창이 노출되어 데이터 입력 및 클릭 흐름을 눈으로 확인할 수 있습니다.
 - ※ 모드 전환 시 세션 동기화를 위해 진행 상황이 초기화되므로 [Step 1: 메뉴 선택]부터 다시 진행해 주세요.

Step 1: 테스트 메뉴 선택

- 상단 메뉴 버튼을 클릭하여 테스트할 기능을 선택합니다.
- 선택한 메뉴의 규격에 맞춰 Step 2(사용자/가맹점) 및 Step 3(파라미터/옵션) 영역이 동적으로 변경됩니다.

4. 메뉴별 상세 가이드

A. 레거시 결제/인증 요청 (간편인증, 소액결제, DCB, DCS)

- Step 2: 사용자 선택
 - Push 알림을 수신할 테스트 대상 회선을 드롭다운에서 선택합니다.

- ※ 테스트 회선 정보 추가나 변경이 필요할 경우 담당자(QA 3팀 김태림)에게 문의 바랍니다.
- **Step 3: 필수 파라미터 확인**
 - 휴대폰 번호, 성명, 식별값, `tAuthKey` 가 기본적으로 자동 완성됩니다.
 - `checkOption` 설정이 필요한 메뉴의 경우, 하단 킷 버튼을 눌러 손쉽게 값을 변경할 수 있습니다:
 - 소액결제: 기존(0), 동의값(1), 동의+가입(2), 이상거래차단(3)
 - DCB결제: 동의값(1), 이상거래차단(3)
 - DCS결제: DCS본인인증(4)
- **주요 동작 버튼:**
 - [**전송 (인증 요청 발송)**]: 웹페이지로 데이터를 주입하고 전송 버튼을 클릭하여 단말로 Push 알림을 발송합니다.
 - [**결과 값 가져오기**]: 서버 통신 결과 JSON 로그를 가져와 하단 텍스트 창에 포매팅하여 노출합니다.

B. PAMS 본인확인 및 2단계 자동 검증

- **Step 2: 사용자 선택 및 회선 타입 설정**
 - 사용자 선택 후 하단 옵션 버튼을 통해 회선 구분을 설정합니다 (`일반회선: false` , `알뜰폰회선: true`).
- **1단계 검증 및 agentTxId 확보:**
 1. [**전송 (인증 요청 발송)**] 클릭 후 단말기에서 인증을 완료합니다.
 2. 반드시 [**결과 값 가져오기**] 버튼을 클릭하여 결과 로그를 수신합니다 (이 과정에서 2단계 계용 `agentTxId` 가 메모리에 자동 저장됩니다).
- **2단계 자동 검증 점프:**
 - [**PAMS 2단계 자동 검증 (URL 점프)**] 버튼을 클릭합니다.
 - QA/상용 환경에 맞는 2단계 검증 URL로 즉시 점프하며, 추출된 `agentTxId` 를 자동 주입하고 확인(`vrifyByOnline`) 처리를 거쳐 최종 JSON 결과를 화면에 표시합니다.

C. 신분증 MPM 대면 인증 테스트 (mpm / a2a)

- **Step 2: 가맹점 선택**
 - `통합 가맹점 (mpm)` 또는 `일반 가맹점 (a2a)` 을 선택합니다 (환경에 맞는 가맹점 ID 자동 매핑).

- **Step 3: MPM 검증 옵션 선택**

- 검증 세부 타입(기본 / 기본+셀피), 가맹점 QR 타입(주민등록증 / 운전면허증 / 전체), 신분증 타입을 선택합니다.

- **주요 동작 버튼:**

- [실시간 가변 QR 코드 및 이미지 추출 가동] : 가변 QR을 생성하고 Base64 이미지를 파싱하여 화면 우측에 실시간 렌더링하며, 유효 타이머 카운트다운을 시작합니다. (단말기 PASS 앱 신분증 스캐너로 촬영)
- [공개키 요청 및 검증하기] : 단말 촬영 후 로딩 상태에서 클릭하면 공개키 요청 및 검증을 완료하고 최종 상태 코드(`verificationStatusCode`)를 출력합니다.

D. 부정가입방지 데모 테스트 (MIS)

- **Step 2: 세부 옵션 선택**

- 인증 수단(실물신분증 촬영 / PASS신분증 이용) 및 신분증 종류(주민등록증 / 운전면허증)를 지정합니다.

- **주요 동작 버튼:**

- [부정가입방지 QR 추출 가동] : MVNO/MNO 토큰 발급, 신원확인 통신, 데이터 복호화를 자동 수행한 후 실시간 생성된 QR 코드와 검증 요청 TX ID를 화면에 출력합니다.
- 단말 인증 완료 후 하단 텍스트 창을 통해 결과를 확인합니다.

5. 자주 묻는 질문 (FAQ) 및 트러블슈팅

Q. 툴 실행이 되지 않아요.

A. 본 툴은 Selenium 기반으로 구동되므로 PC 내에 **Google Chrome 브라우저**가 설치되어 있어야 합니다. Chrome 설치 여부를 확인해 주세요.

Q. 툴이 너무 늦게 켜져요.

A. 백그라운드에서 웹 드라이버 엔진을 초기화하고 사전 보안/네트워크 연결을 구성하기 때문에 최초 기동 시 2~3초간 준비 시간이 걸립니다. `.exe` 파일은 한 번만 실행하고 기다려 주세요.

Q. 툴에서는 성공이라고 떴는데 단말로 Push가 오지 않아요.

A. 선택한 사용자의 번호/통신사 일치 여부를 먼저 확인해 주세요. 일치함에도 오지 않는다면 사내 방화벽이나 통신사 게이트웨이 이슈로 백엔드 단에서 드롭(차단)되었을 수 있습니다. 브라우저 노출 모드(디버그)로 전환하여 웹에서 직접 확인해 보시거나 담당자에게 문의 바랍니다.

Q. 버튼이 눌리지 않거나 결과 값이 나오지 않아요.

A. 상용/QA 망의 응답 지연으로 웹 로딩이 길어졌거나 오류 Alert 팝업이 뜬 상태일 수 있습니다. 잠시 후 재시도해 주시고, 지속될 경우 디버그 모드(Web 노출)로 전환하여 웹 에러 메시지를 확인해 주세요.

Q. MPM 검증 버튼을 눌렀을 때 "서버 거절" 팝업이 뜹니다.

A. 단말기에서 QR 스캔 및 본인확인 처리를 완료하기 전에 [공개키 요청 및 검증하기]를 눌러거나 유효시간(3분)이 만료된 경우입니다. 단말기 진행 상태를 확인한 후 다시 시도해 주세요.

Q. PAMS 2단계 버튼 클릭 시 agentTxId 누락 에러가 발생합니다.

A. PAMS 2단계 검증을 위해서는 1단계 전송 후 [🔍 결과 값 가져오기] 버튼을 먼저 눌러 1단계 JSON 조회를 완료해야 합니다. 순서대로 진행해 주세요.

▼ 🛠️ [기술 리뷰] 상용 환경 대응 하드닝(Hardening) 및 성능 최적화 보고서

💡 개요

- **작성자:** QA 3팀 김태림[cite: 19, 28]
- **목적:** 상용 환경 배포 및 실무 적용 과정에서 발생한 기술적 결함(UI 프리징, DOM 이벤트 미동기화, 보안 정책 예외, 상태 머신 오염)의 원인 분석 및 해결 과정 기록[cite: 19, 23, 24, 26]
- **적용 버전:** ver 0.1 ~ 0.5 (초기 버전) → ver 0.85 (하드닝 완료)[cite: 19, 28]

1. 하드닝 요약 (Before vs After)

영역	초기 구현 (As-Is / v0.5)[cite: 28]	하드닝 적용 (To-Be / v0.85)[cite: 19]	달성 성과 및 이득
GUI 반응성	메인 스레드 블로킹으로 인한 Windows '응답 없음' 발생[cite: 19]	<code>threading.Thread</code> 기반 완전 비동기 워커 스레드 분리[cite: 19]	UI 프리징 제로화, 백그라운드 작업 중 실시간 시계/타이머 유지[cite: 19]
PAMS 검증	상용 환경에서 폼 값 미인식 버튼 씹힘 및 2단계 점프 실패[cite: 25]	이벤트 체인 강제 디스패치 + JS 인라인 함수 직접 호출 폴백[cite: 25]	상용 폼 유효성 검사 우회 통과, 2단계 점프 검증 성공률 100% 달성[cite: 25]
MPM 대면검증	미인증/시간 만료 시 브라우저 Alert	<code>UnexpectedAlertPresentException</code> 방어 및 상태 코드 폴링[cite: 24]	불필요한 고정 딜레이 제거, Alert 텍스트 파싱을 통

영역	초기 구현 (As-Is / v0.5)[cite: 28]	하드닝 적용 (To-Be / v0.85)[cite: 19]	달성 성과 및 이득
	로 인한 크래시 발생[cite: 24]		한 예외 피드백[cite: 24]
MIS 부정가입	크로스 도메인 캔버스 추출 시 <code>SecurityError</code> 발생[cite: 23]	JS 레벨 <code>try/catch</code> 가드 및 <code>img[src]</code> Base64 다중 폴백[cite: 23]	Tainted Canvas 보안 제약 우회, 실시간 QR 및 TX ID 안정적 확보[cite: 23]
상태 머신	이전 테스트의 TX ID가 메모리에 잔류하여 오검증 위험[cite: 26]	모든 테스트 액션 진입 시 트랜잭션 변수 강제 초기화[cite: 26]	이전 잔류 데이터 참조로 인한 거짓 양성(False Positive) 원천 차단[cite: 26]
UI/UX 안정성	메뉴 전환 시 위젯 파괴/재배치로 인한 화면 깜빡임 및 점핑[cite: 19]	기본 컨테이너 고정 및 메뉴별 선택적 라우팅 스위칭[cite: 19]	렌더링 부하 최소화 및 빠릿하고 정적인 레이아웃 전환 완성[cite: 19]

2. 세부 문제 분석 및 기술적 해결 내역

1) GUI 메인 루프 프리징 및 스레드 병목 해결

- 문제점 (As-Is):
 - Tkinter/CustomTkinter는 단일 메인 스레드로 이벤트 루프를 처리함[cite: 19].
 - Selenium의 DOM 탐색(`WebDriverWait`), 네트워크 대기, 페이지 로드 등 블로킹 I/O 작업이 메인 스레드에서 직접 실행되어, 웹 통신 중 창 이동 불가, 시계 멈춤, Windows "응답 없음" 팝업이 발생함[cite: 19].
- 해결책 (To-Be):
 - `run_async(target_func, *args, **kwargs)` 래퍼 함수를 도입하여 모든 웹 통신 및 브라우저 제어 작업을 데몬 스레드로 격리 실행함[cite: 19].
 - UI 업데이트(메시지박스, 텍스트 삽입, 타이머 조작)는 `self.root.after(0, callback)` 구조를 엄격히 준수하여 스레드 안전성(Thread-Safety)을 확보함[cite: 19].
- 개선 효과:
 - 장시간 소요되는 상용 네트워크 통신 중에도 GUI 컨트롤이 부드럽게 유지되며, 실시간 타이머와 시계가 끊김 없이 갱신됨[cite: 19].

2) PAMS 상용 환경 이벤트 불일치 및 2단계 자동화 안정화

- 문제점 (As-Is):

- 상용 페이지의 프론트엔드 프레임워크가 순수 키보드 입력(`send_keys`)만으로는 입력 변경 이벤트를 감지하지 못해, 값은 채워졌으나 실제 유효성 검증을 통과하지 못해 전송 버튼이 비활성화되거나 씹히는 문제 발생[cite: 25].
- 2단계 점프 후 검증 버튼(`btn1`)의 DOM 렌더링 방식 차이로 인해 물리적 클릭이 실패하거나 비활성 상태로 남아 결과 조회가 중단됨[cite: 25].

- 해결책 (To-Be):

- `mobileDrvLcnseVrifyHistId3` 필드에 값 주입 후, 브라우저 엔진에 `input`, `change`, `keyup`, `blur` 이벤트를 강제 디스패치(Dispatch)하여 상용 유효성 체크를 즉시 통과시킴[cite: 25].
- 버튼 클릭 로직을 3중 구조(다중 XPath 매칭 → 뷰포트 중앙 스크롤 `scrollIntoView` → 미발견 시 내부 JS 함수 `vrifyByOnline()` 직접 호출)로 폴백 처리함[cite: 25].

- 개선 효과:

- 프론트엔드 UI 렌더링 지연이나 버튼 가림 현상과 무관하게 상용 2단계 검증이 백그라운드에서 즉시 완료됨[cite: 25].

3) 단말 미인증 시 Alert 블로킹 및 상태 코드 폴링 최적화 (MPM)

- 문제점 (As-Is):

- 단말기에서 QR 스캔 및 본인인증을 마치지 않은 상태에서 검증 버튼을 누를 경우, 웹 서버는 응답 필드를 채우지 않고 브라우저 Native Alert("인증 미완료")를 띄움[cite: 8, 24].
- 이 상태에서 Selenium이 DOM 조회를 지속 시도하다가 `UnexpectedAlertPresentException` 을 던지며 10초 타임아웃까지 툴이 멈추거나 튕기는 결함 발생[cite: 8, 24].
- 검증 결과 확인 시 단순 `time.sleep(1.5)` 방식에 의존하여 상용망 통신 지연 시 빈 문자열(`""`)을 성공으로 오판독하는 결함 존재[cite: 15, 24].

- 해결책 (To-Be):

- 상태 코드 필드(`verificationStatusCode`)를 모니터링할 때 `UnexpectedAlertPresentException` 전용 예외 처리 블록을 배치, Alert 발생 시 브라우저 알림 텍스트를 파싱하여 사용자에게 "서버 거절: [사유]" 팝업으로 즉시 피드백함[cite: 24].
- 단순 슬립을 제거하고, 상태 코드 필드에 실제 데이터가 들어올 때까지 능동적으로 감지하는 조건부 폴링(`WebDriverWait.until`)으로 전환함[cite: 24].

- 개선 효과:

- 비정상 시나리오(네거티브 케이스)에서도 도구가 중단되지 않고 명확한 실패 사유를 안내하며, 상용망 지연 시에도 상태 코드를 누락 없이 정확하게 파싱함[cite: 24].

4) Tainted Canvas 보안 제약 우회 및 실측 TX ID 추출 (MIS)

- 문제점 (As-Is):

- 상용 도메인(`mis-mobileid.com`)에서 부정가입방지 QR을 캔버스(`canvas`) 태그로 렌더링할 때, 외부 CDN 리소스 오염(Tainted Canvas)으로 인해 `toDataURL()` 호출 시 브라우저 레벨의 `SecurityError: The operation is insecure` 예외가 발생하며 스크립트 실행이 중단됨[cite: 14, 23].
- 거래 번호(TX ID)를 더미 타임스탬프로 생성하던 방식이어서 실제 단말 통신 추적이 불가능했음[cite: 14, 23].

- 해결책 (To-Be):

- JavaScript 실행 구문 내부에 브라우저 레벨의 `try-catch` 가드를 추가하여, 캔버스 보안 에러 발생 시 즉시 `img[src]` 태그의 Base64 데이터를 탐색하도록 다중 폴백 구조를 구현함[cite: 23].
- HTML 내 실제 input 필드(`txId`)의 value 속성을 직접 추출하는 파서를 도입하여 실측 TX ID를 GUI에 동기화함[cite: 23].

- 개선 효과:

- 브라우저 보안 샌드박스 정책에 구애받지 않고 QR 이미지와 실제 통신 TX ID를 100% 신뢰도로 획득할 수 있게 됨[cite: 23].

5) False Positive(거짓 양성) 방지를 위한 트랜잭션 수명주기 격리

- 문제점 (As-Is):

- 이전 테스트 사이클에서 조회된 `last_agent_tx_id` 가 새 테스트 수행 시에도 메모리에 잔류함[cite: 17, 26].
- 네트워크 장애나 요청 실패로 인해 이번 요청이 정상 발송되지 않았음에도, 이전 테스트의 잔류 ID로 2단계 검증이 진행되어 "성공"으로 잘못 판정되는 치명적인 오검증(False Positive) 위험 존재[cite: 17, 26].

- 해결책 (To-Be):

- `execute_send` , `execute_extract_qr_data` , `execute_mis_qr_logic` 등 모든 신규 트랜잭션 진입 시점 최상단에 `self.last_agent_tx_id = None` 을 강제 호출하여 상태 머신을 리셋함[cite: 26].

- 개선 효과:

- 1단계 통신 성공 및 JSON 파싱이 정상 완료된 건에 대해서만 2단계 점프가 활성화되어 자동화 검증의 무결성을 보장함[cite: 25, 26].

3. 정량적/정성적 성과 (ROI)

1) 정량적 성과

- 테스트 사이클 소요 시간 단축:
 - 폼 수기 입력 및 도메인 전환: 건당 약 **1분 30초** 소요[cite: 28]
 - 도구 원클릭 자동 주입 및 발송: 건당 **5초 이내** 완료[cite: 19, 22]
 - 회귀 테스트(Regression Test) 진행 시 1회 사이클당 소요 공수 약 **85% 이상 절감**
- 결과 판독 정확도:
 - 눈으로 브라우저 응답을 찾던 수기 검증 방식에서 JSON 파싱 및 상태 코드 자동 판별로 전환하여 **휴먼 에러 0% 달성**[cite: 19, 24, 26]

2) 정성적 성과 (QA 엔지니어링 관점)

- 상용/QA 통합 하드닝 레퍼런스 구축: 단순한 브라우저 매크로 수준을 넘어, 상용 환경의 지연 시간, 비동기 Alert, 보안 샌드박스 예외를 완벽히 통제하는 견고한 아키텍처 수립 [cite: 23, 24, 26].
- 배포 및 유지보수 편의성 극대화: **.exe** 단일 바이너리 배포 방식을 채택하여 팀 내 비개발 직군 테스터도 환경 설정(Python, 의존성 패키지 설치) 없이 즉시 실무에 투입 가능 [cite: 28].